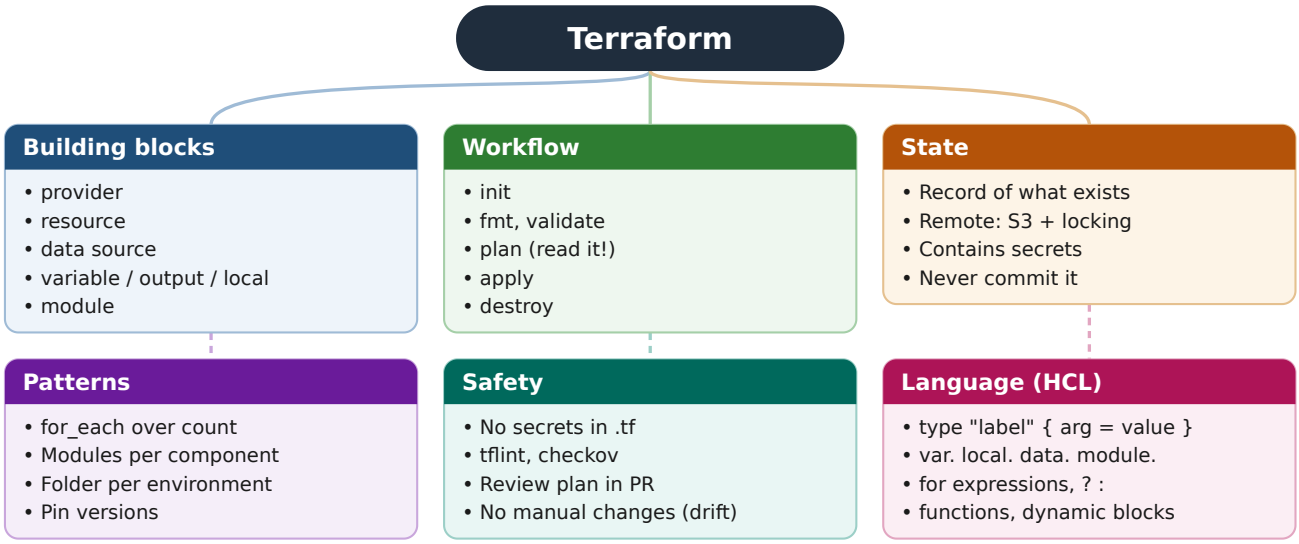
9. Terraform (Infrastructure as Code)

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 9 of 18 · Thirupathi Keloth · v2.0

Contents

9.0 Why Infrastructure as Code
9.1 IaC concepts and the tool landscape
9.2 How Terraform works
9.3 Installation and CLI
9.4 The Terraform language (HCL)
9.5 Providers in depth
9.6 State in depth
9.7 Data sources, outputs and locals
9.8 Modules in depth
9.9 Lifecycle, refactoring and checks
9.10 Environments: workspaces vs directories
9.11 Project structure and conventions
9.12 Team workflow and Terraform in CI/CD
9.13 Terraform security
9.14 Testing Terraform
9.15 Importing existing infrastructure
9.16 Complete project: modular VPC + load-balanced web tier
9.17 Troubleshooting
9.18 Interview questions and answers
9.19 Self-check: I can...



9.0 Why Infrastructure as Code

Clicking resources together in a cloud console is slow, error-prone, undocumented and impossible to reproduce exactly. **Infrastructure as Code (IaC)** describes infrastructure in text files that are versioned in Git, reviewed in pull requests, tested in CI and applied automatically.

Benefit	What it means in practice
Repeatable	Build identical dev, staging and prod; rebuild a region for DR
Reviewable	Changes go through pull requests and `plan` output
Auditable	Git history shows who changed what and why
Fast	Whole environments in minutes

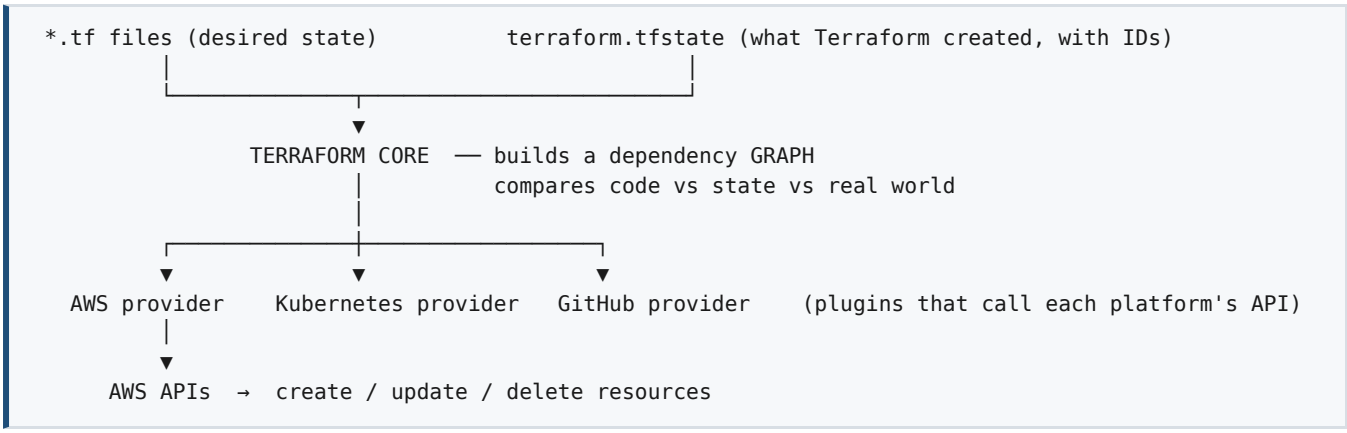
Benefit	What it means in practice
Self-documenting	The code *is* the description of the infrastructure
Safer	Drift detection, policy checks, no "snowflake" servers

9.1 IaC concepts and the tool landscape

Concept	Meaning
Declarative	You describe the desired end state ("3 subnets exist"); the tool works out the steps (Terraform, CloudFormation, Kubernetes YAML)
Imperative	You write the steps ("create subnet, then…") (scripts, AWS CLI)
Idempotent	Applying the same code again changes nothing if reality already matches
Drift	Real infrastructure differs from code (someone changed it manually)
Immutable infrastructure	Replace resources instead of modifying them in place (new AMI/image → new instances)
Provisioning vs configuration	Creating infrastructure (Terraform) vs configuring inside servers (Ansible)

Tool	Type	Notes
Terraform	Declarative, multi-cloud, HCL	Most popular IaC tool; huge provider ecosystem; Business Source License since 2023
OpenTofu	Open-source fork of Terraform (Linux Foundation)	Drop-in compatible CLI (`tofu`), same HCL
CloudFormation / CDK	AWS-native	Managed state; CDK uses programming languages
Pulumi	Real programming languages	TypeScript/Python/Go for infrastructure
Crossplane	Kubernetes-native IaC	Cloud resources as Kubernetes objects
Ansible	Configuration management (can provision too)	Procedural playbooks; complements Terraform

9.2 How Terraform works



The core workflow:

```
write code → terraform init → terraform plan → review → terraform apply → (later) destroy
```

Step	What happens
`init`	Downloads providers and modules, configures the backend (state location), writes `.terraform.lock.hcl`
`plan`	Refreshes current state from the real APIs, compares with the code, prints the change set
`apply`	Executes the plan in dependency order (in parallel where possible), updates the state
`destroy`	Deletes everything the configuration manages

Plan symbols: `+` create · `~` update in place · `-` destroy · `-/+` destroy then create (**replace**) · `+/-` create then destroy (with `create_before_destroy`) · `<=` read (data source). Always read *why* something will be replaced ("forces replacement").

9.3 Installation and CLI

Install: download the binary from HashiCorp (or `apt` / `dnf` from the HashiCorp repository, `brew install terraform`), or use a version manager (`tfenv`) so each project uses its pinned version. Check: `terraform version`.

Command	Purpose
`terraform init`	Initialize (run after cloning, adding providers/modules, or changing the backend). `-upgrade` updates providers within constraints; `-reconfigure` / `-migrate-state` for backend changes
`terraform fmt -recursive`	Format code to the standard style
`terraform validate`	Syntax and internal consistency check
`terraform plan`	Preview changes; `-out=tfplan` saves the exact plan
`terraform apply` / `apply tfplan`	Apply (asks for confirmation) / apply a saved plan exactly
`terraform destroy`	Delete everything in this configuration
`terraform output [-json] [name]`	Show outputs
`terraform show`	Show state or a saved plan in human-readable form
`terraform state list / show / mv / rm / pull`	Inspect and manipulate state (see 9.6)
`terraform import ADDRESS ID`	Bring an existing resource under management (prefer `import` blocks, 9.15)
`terraform console`	Evaluate expressions interactively
`terraform graph`	Dependency graph (DOT format)
`terraform providers`	Providers used by the configuration and modules
`terraform workspace list/new/select`	Workspaces (9.10)
`terraform force-unlock LOCK_ID`	Remove a stuck state lock (only when sure nobody is running)

Command	Purpose
`terraform test`	Run `.tftest.hcl` tests (9.14)
`terraform login`	Authenticate to HCP Terraform / a private registry

Useful flags	Meaning
`-var="env=prod"`, `-var-file=prod.tfvars`	Set variables
`-replace=aws_instance.web`	Force replacement of one resource
`-refresh-only`	Update state to match reality without changing infrastructure (drift review)
`-target=module.vpc`	Limit to some resources: **emergency use only**, can leave state inconsistent
`-auto-approve`	Skip confirmation (CI only, after a reviewed plan)
`-lock-timeout=5m`	Wait for a state lock
`-parallelism=10`	Concurrent operations (default 10)
`-input=false`	Never prompt (CI)

Environment variable	Use
`TF_VAR_name`	Set input variable `name` (good for secrets in CI)
`TF_LOG=DEBUG`, `TF_LOG_PATH=tf.log`	Debug logging (provider API calls)
`TF_CLI_ARGS_plan="-parallelism=20"`	Default flags per command
`TF_IN_AUTOMATION=1`	Cleaner output in CI
`AWS_PROFILE`, `AWS_REGION`	Credentials/region for the AWS provider

Files Terraform creates:

File/folder	Commit to Git?
`*.tf`, `*.tftest.hcl`	Yes
`.terraform.lock.hcl` (provider versions + checksums)	**Yes**
`.terraform/` (downloaded providers/modules)	No
`terraform.tfstate`, `*.tfstate.backup`	**Never** (remote backend instead)
`*.tfvars` with secrets/env specifics	Usually no (or only non-secret ones)
`tfplan` files	No (they can contain secrets)

9.4 The Terraform language (HCL)

Anatomy of a block

```
# block type = resource | label 1 = resource type | label 2 = your name for it
resource "aws_instance" "web" {
  ami           = "ami-0abc123"     # ARGUMENT: name = value
  instance_type = var.instance_type # a value can reference other things
  tags = {                          # map value
    Name = "web-1"
  }
  root_block_device { # NESTED BLOCK (no "=")
    volume_size = 20
  }
}
```

- **Block** = `type "labels" { body }`. **Argument** = `name = value`. **Nested block** = `name { ... }` without `=`.
- The resource's **address** is `aws_instance.web`. Other code reads its attributes, e.g. `aws_instance.web.public_ip`.
- Terraform reads **all** `.tf` **files in the folder** together; file names are only for humans.

Block types

Block	Purpose	Example
`terraform`	Terraform version, required providers, backend (state location)	`terraform { required_version = ">= 1.6" }`
`provider`	Configures a platform's API (region, credentials from environment/role)	`provider "aws" { region = "ap-south-1" }`
`resource`	Something Terraform **creates and manages**	`resource "aws_s3_bucket" "logs" { ... }`
`data`	**Reads** something that already exists	`data "aws_availability_zones" "available" {}`
`variable`	An **input**	`variable "env" { type = string }`
`locals`	**Computed helper values** inside the module	`locals { name = "${var.env}-web" }`
`output`	A value printed after apply / used by other modules	`output "ip" { value = aws_instance.web.public_ip }`
`module`	Uses a reusable folder of Terraform code	`module "vpc" { source = "./modules/vpc" }`
`moved` / `import`	Refactor without destroying / adopt existing resources	`import` block with `to = aws_s3_bucket.logs` and `id = "my-bucket"`

How to reference things

You write	It means
`var.env`	Input variable `env`
`local.name`	Local value `name`
`aws_vpc.main.id`	Attribute `id` of resource `aws_vpc` named `main`
`data.aws_ami.ubuntu.id`	Attribute of a data source

You write	It means
`module.vpc.vpc_id`	Output `vpc_id` of module `vpc`
`aws_subnet.private[0].id`	Item 0 when the resource uses `count`
`aws_subnet.this["public-a"].id`	Item by key when the resource uses `for_each`
`each.key` / `each.value`	Current item inside a `for_each` resource
`count.index`	Current number (0, 1, 2…) inside a `count` resource
`path.module`	Folder of the current module (for files/templates)

Implicit dependencies: referencing `aws_vpc.main.id` inside a subnet tells Terraform to create the VPC first. Use `depends_on` only when there is no reference.

Data types

Type	Example
`string`	`"t3.micro"`
`number`	`3`
`bool`	`true`
`list(string)`	`["a", "b"]` — ordered, accessed by index `[0]`
`set(string)`	`toset(["a", "b"])` — unique, no order
`map(string)`	`{ env = "dev", team = "ops" }` — accessed by key `["env"]`
`object({...})`	`{ name = string, size = number }` — fixed structure

Variables in detail

```
variable "instance_type" {
  description = "EC2 size for the web tier" # documents the input
  type        = string                      # type constraint
  default     = "t3.micro"                  # optional: no default = required input

  validation { # reject bad values early
    condition     = contains(["t3.micro", "t3.small"], var.instance_type)
    error_message = "Use t3.micro or t3.small."
  }
}

variable "db_password" {
  type      = string
  sensitive = true # hidden in plan/apply output (still stored in state!)
}
```

Ways to set variables (later ones win): `default` → `terraform.tfvars` / `*.auto.tfvars` → `-var-file=prod.tfvars` → `-var="env=prod"`. Environment variable form: `TF_VAR_db_password`.

Expressions

```
locals {
  name        = "${var.project}-${var.env}"                    # string interpolation
  is_prod     = var.env == "prod"                              # comparison → bool
  size        = local.is_prod ? "t3.small" : "t3.micro"        # conditional: cond ? a : b
  upper_names = [for s in var.servers : upper(s)]              # for expression → list
  name_map    = { for s in var.servers : s => "${s}.internal" } # for expression → map
  subnet_ids  = aws_subnet.private[*].id                       # splat: every id
}
```

Useful built-in functions

Function	Example	Result
`length`	`length(["a","b"])`	`2`
`upper` / `lower`	`upper("dev")`	`"DEV"`
`join` / `split`	`join(",", ["a","b"])`	`"a,b"`
`merge`	`merge(local.tags, { Name = "web" })`	Combined map
`concat`	`concat(["a"], ["b"])`	`["a","b"]`
`lookup`	`lookup(var.sizes, "prod", "t3.micro")`	Value or default
`contains`	`contains(["a","b"], "a")`	`true`
`cidrsubnet`	`cidrsubnet("10.0.0.0/16", 8, 1)`	`"10.0.1.0/24"`
`file` / `templatefile`	`templatefile("${path.module}/init.sh.tftpl", { port = 80 })`	File content (with variables)
`try` / `coalesce`	`try(var.cfg.size, "small")`	Fallback when missing
`toset` / `tolist`	`toset(var.names)`	Type conversion

Try any function in `terraform console`.

Meta-arguments (work on every resource)

Meta-argument	Purpose
`count = 3`	Create N copies (`count.index`)
`for_each = toset([...])` or a map	One copy per item (`each.key` , `each.value`) — preferred
`depends_on = [aws_iam_role.x]`	Force an order when there is no reference
`provider = aws.us_east`	Use an aliased provider (e.g. second region)
`lifecycle { ... }`	`create_before_destroy` , `prevent_destroy` , `ignore_changes`

Dynamic block (repeat a nested block from a list):

```
variable "ingress_ports" {
  type    = list(number)
  default = [80, 443]
}

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Complete annotated example: VPC + public subnet + web server (single file)

```
# ---------- versions.tf ----------
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws" # where to download the provider
      version = "~> 5.0"        # any 5.x version
    }
  }
}

provider "aws" {
  region = var.region # credentials come from your AWS CLI profile / IAM role
  default_tags {
    tags = local.common_tags # added to every resource automatically
  }
}

# ---------- variables.tf ----------
variable "region" {
  type    = string
  default = "ap-south-1"
}

variable "env" {
  type        = string
  description = "Environment name, e.g. dev or prod"
}

variable "my_ip_cidr" {
  type        = string
  description = "Your public IP for SSH, e.g. 203.0.113.10/32"
}

# ---------- main.tf ----------
locals {
  name        = "lab-${var.env}"
  common_tags = { Project = "devops-lab", Env = var.env, ManagedBy = "terraform" }
}

data "aws_availability_zones" "available" {
  state = "available" # read the AZ names of this region
}

data "aws_ami" "ubuntu" { # find the newest Ubuntu 24.04 image
  most_recent = true
  owners      = ["099720109477"] # Canonical's account
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id # reference → VPC is created first
}

resource "aws_subnet" "public" {
```

```
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8, 1) # 10.0.1.0/24
  availability_zone       = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true
  tags                    = { Name = "${local.name}-public-a" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"                 # everything else ...
    gateway_id = aws_internet_gateway.igw.id # ... goes to the internet gateway
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name   = "${local.name}-web"
  vpc_id = aws_vpc.main.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "SSH only from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip_cidr]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1" # all protocols
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data              = <<-EOT
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOT
  tags                   = { Name = "${local.name}-web" }
}

# ---------- outputs.tf ----------
output "web_url" {
  description = "Open this in a browser"
  value       = "http://${aws_instance.web.public_ip}"
}
```

Run it: `terraform init` → `terraform plan -var="env=dev" -var="my_ip_cidr=<your-ip>/32"` → `terraform apply ...` → open the URL → **`terraform destroy ...`** when finished.

Naming conventions: snake_case names (`aws_instance.web_server`), one purpose per file (`versions.tf`, `variables.tf`, `main.tf`, `outputs.tf`), `this` as the name when a module has only one resource of a type, run `terraform fmt` before every commit.

9.5 Providers in depth

A **provider** is a plugin that knows one platform's API: `hashicorp/aws`, `hashicorp/azurerm`, `hashicorp/google`, `hashicorp/kubernetes`, `hashicorp/helm`, `integrations/github`, `hashicorp/random`, `hashicorp/tls`, `cloudflare/cloudflare`, `grafana/grafana`, `datadog/datadog`… thousands are on registry.terraform.io.

Version constraints and the lock file

```
terraform {
  required_version = ">= 1.6, < 2.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.70" # >= 5.70, < 6.0
    }
    random = {
      source  = "hashicorp/random"
      version = ">= 3.6"
    }
  }
}
```

Constraint	Means
`= 5.70.0`	Exactly
`>= 5.70`	At least
`~> 5.70`	Any 5.x from 5.70 (no 6.0)
`~> 5.70.0`	Any 5.70.x patch

`.terraform.lock.hcl` records the exact versions and checksums chosen by `init`: commit it so every machine and CI uses identical providers; update deliberately with `terraform init -upgrade`.

Configuring the AWS provider

```
provider "aws" {
  region = "ap-south-1"

  default_tags {
    tags = {
      Project   = "shop"
      ManagedBy = "terraform"
    }
  }
}

# Second provider configuration for another region (alias)
provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"
}

resource "aws_acm_certificate" "cdn" {
  provider          = aws.us_east_1 # CloudFront certificates must live in us-east-1
  domain_name       = "shop.example.com"
  validation_method = "DNS"
}

# Deploy into another account by assuming a role
provider "aws" {
  alias  = "prod"
  region = "ap-south-1"
  assume_role {
    role_arn     = "arn:aws:iam::222222222222:role/terraform-deploy"
    session_name = "terraform"
  }
}
```

Authentication (never hard-code keys in `.tf` files): environment variables, `AWS_PROFILE` / SSO, an instance/container role, or **OIDC from CI** (GitHub Actions assumes a deploy role). Terraform uses the same credential chain as the AWS CLI.

9.6 State in depth

What state is and why it matters

`terraform.tfstate` is a JSON file mapping each resource address in your code (`aws_instance.web`) to the **real object ID** (`i-0abc…`) and its last known attributes. Terraform needs it to know what it manages, compute diffs, and track dependencies and metadata.

State is sensitive: it can contain passwords, private keys and connection strings in plain text (even for `sensitive` values). Protect it like a secret.

Local vs remote state

	Local (`terraform.tfstate` on disk)	Remote backend
Teamwork	Impossible (everyone has a different copy)	Shared, single source of truth
Locking	None	Prevents two applies at once
Safety	Lost laptop = lost state	Versioned, encrypted, backed up
Use	Personal experiments only	**Everything real**

S3 backend (AWS)

```
terraform {
  backend "s3" {
    bucket       = "acme-terraform-state-123456789012"
    key          = "prod/network/terraform.tfstate" # one key per stack/environment
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true # native S3 locking (Terraform 1.10+); older setups used a DynamoDB table
  }
}
```

State bucket requirements: versioning on (recover old states), encryption (SSE-KMS), Block Public Access, a bucket policy limiting access to the Terraform roles, ideally in a separate "tooling" account. Create the bucket once (bootstrap) with a small separate configuration or the CLI. Backend blocks can't use variables; pass values with `terraform init -backend-config=backend-prod.hcl` for multiple environments.

Other backends: HCP Terraform / Terraform Enterprise (remote runs, policy, UI), `gcs`, `azurerm`, `kubernetes`, `pg`.

State commands

Command	Use
`terraform state list`	All managed resource addresses
`terraform state show aws_vpc.main`	Attributes of one resource
`terraform state mv aws_instance.web aws_instance.app`	Rename/move in state (prefer `moved` blocks in code)
`terraform state rm aws_s3_bucket.old`	Stop managing a resource **without deleting it** (prefer `removed` blocks)
`terraform state pull > backup.tfstate`	Download the current state (backup before surgery)
`terraform state push`	Upload a state file (dangerous; last resort)
`terraform apply -refresh-only`	Accept real-world changes into state after review

Drift

Someone changed a security group in the console → next `plan` shows Terraform wants to revert it. Decide: **revert** (apply the code) or **adopt** (change the code to match). Prevent drift with least-privilege console access, scheduled `plan` jobs that alert on drift, and AWS Config.

Splitting state

One giant state = slow plans and a huge blast radius. Split by **environment** and **component/layer** (network, data, compute, DNS), each with its own state key and pipeline. Share values between stacks with outputs read through `terraform_remote_state`, data sources (look resources up by tags), or SSM parameters.

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "acme-terraform-state-123456789012"
    key    = "prod/network/terraform.tfstate"
    region = "ap-south-1"
  }
}

locals {
  private_subnet_ids = data.terraform_remote_state.network.outputs.private_subnet_ids
}
```

9.7 Data sources, outputs and locals

Construct	Purpose	Example
Data source	Read existing information (not managed by this config)	Latest AMI, account ID, existing VPC by tag, IAM policy documents, secrets
Output	Expose values: CLI display, CI, other stacks, parent modules	VPC ID, ALB DNS name
Local	Name an expression to avoid repetition	Common tags, name prefixes

```
data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

data "aws_vpc" "shared" {
  tags = { Name = "shared-vpc" }
}

data "aws_iam_policy_document" "read_assets" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["arn:aws:s3:::shop-assets/*"]
  }
}

data "aws_secretsmanager_secret_version" "db" {
  secret_id = "shop/prod/db"
}

locals {
  account_id = data.aws_caller_identity.current.account_id
  name       = "shop-${var.env}"
}

output "account_id" {
  value = local.account_id
}

output "db_connection" {
  value     = jsondecode(data.aws_secretsmanager_secret_version.db.secret_string)["host"]
  sensitive = true # hidden in CLI output (still stored in state)
}
```

`aws_iam_policy_document` builds IAM JSON in HCL (validated, composable) instead of hand-written JSON strings.

9.8 Modules in depth

A **module** is a folder of `.tf` files with inputs (variables) and outputs. Every configuration is a module (the **root module**); modules it calls are **child modules**.

```
modules/network/
├── main.tf         # resources
├── variables.tf    # inputs (typed, documented, validated)
├── outputs.tf      # outputs
├── versions.tf     # required_providers (no provider blocks inside reusable modules)
└── README.md       # usage example, inputs/outputs table
```

Calling modules

```
module "network" {
  source = "./modules/network" # local path

  name       = "shop-dev"
  cidr_block = "10.10.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
}

module "vpc_community" {
  source  = "terraform-aws-modules/vpc/aws" # public registry
  version = "~> 5.13"                       # ALWAYS pin module versions

  name            = "shop-prod"
  cidr            = "10.20.0.0/16"
  azs             = ["ap-south-1a", "ap-south-1b"]
  public_subnets  = ["10.20.1.0/24", "10.20.2.0/24"]
  private_subnets = ["10.20.11.0/24", "10.20.12.0/24"]
}

# Git source pinned to a tag:
# source = "git::https://github.com/acme/terraform-modules.git//network?ref=v1.4.0"

output "vpc_id" {
  value = module.network.vpc_id
}
```

Source type	Example
Local path	`./modules/network`
Terraform Registry	`terraform-aws-modules/vpc/aws` + `version`
Git	`git::https://...//subdir?ref=v1.4.0` (pin a tag/commit)
S3 / HTTP archive	`s3::https://...module.zip`
Private registry	`app.terraform.io/acme/network/aws`

Good module design: one clear purpose · minimal required inputs with sensible defaults · typed and validated variables · useful outputs · no hard-coded regions/accounts · no `provider` blocks (the caller passes providers) · semantic versioning with a changelog · examples and tests. Don't wrap a single resource in a module without adding value.

9.9 Lifecycle, refactoring and checks

lifecycle

```
resource "aws_db_instance" "main" {
  # ...
  lifecycle {
    prevent_destroy = true # plan fails if anything would destroy it
  }
}

resource "aws_launch_template" "web" {
  # ...
  lifecycle {
    create_before_destroy = true # new one first, then delete the old (no gap)
  }
}

resource "aws_autoscaling_group" "web" {
  # ...
  lifecycle {
    ignore_changes = [desired_capacity] # changed by autoscaling at runtime: don't fight it
  }
}

resource "aws_instance" "app" {
  # ...
  lifecycle {
    replace_triggered_by = [aws_launch_template.web.latest_version]
    precondition {
      condition     = var.instance_type != "t2.micro"
      error_message = "t2 instances are not allowed; use t3/t4g."
    }
  }
}
```

Refactoring without destroying: moved, import, removed

```
moved {
  from = aws_instance.web
  to   = aws_instance.app # renamed in code: state follows, nothing is recreated
}

moved {
  from = aws_subnet.public[0]
  to   = aws_subnet.public["a"] # switched from count to for_each
}

import {
  to = aws_s3_bucket.legacy
  id = "acme-legacy-bucket" # adopt an existing bucket (see 9.15)
}

removed {
  from = aws_s3_bucket.archive # stop managing it, keep the real bucket
  lifecycle {
    destroy = false
  }
}
```

check blocks (continuous assertions)

```
check "website_up" {
  data "http" "site" {
    url = "https://shop.example.com/health"
  }
  assert {
    condition     = data.http.site.status_code == 200
    error_message = "Health endpoint is not returning 200."
  }
}
```

Checks produce **warnings** on plan/apply without blocking: useful for post-deployment validation.

9.10 Environments: workspaces vs directories

Approach	How	Pros	Cons
Directory per environment	`envs/dev`, `envs/prod` each with its own backend key and tfvars, calling shared modules	Explicit, different settings/versions per env, separate permissions	Some duplication
Workspaces	`terraform workspace new prod`; same code, separate state per workspace; `terraform.workspace` in expressions	Little duplication	Easy to apply to the wrong workspace; same backend/permissions; hidden differences
Wrapper tools	Terragrunt (DRY backends and inputs), Terramate	Large estates	Extra tooling

Most teams use **directories (or separate stacks) per environment** with shared versioned modules; workspaces suit short-lived copies (feature environments).

9.11 Project structure and conventions

```
infrastructure/
├── modules/
│   ├── network/          (VPC, subnets, routes, NAT, endpoints)
│   ├── web-service/      (ALB, target group, ASG, launch template, security groups)
│   └── database/         (RDS, subnet group, parameter group, secret)
├── envs/
│   ├── dev/
│   │   ├── backend.tf    (state key: dev/...)
│   │   ├── main.tf       (module calls)
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   │   └── outputs.tf
│   └── prod/ ...
├── .tflint.hcl
├── .pre-commit-config.yaml
└── README.md
```

Conventions: `snake_case` names · resource name describes the role (`aws_security_group.web`, not `sg1`) · `this` for a module's single main resource · one purpose per file (`versions.tf`, `providers.tf`, `variables.tf`, `outputs.tf`, `main.tf`, `data.tf`) · descriptions on every variable and output · consistent

tags via `default_tags` · `terraform fmt` + `validate` + `tflint` before every commit · generate module docs with `terraform-docs`.

9.12 Team workflow and Terraform in CI/CD

```
feature branch → PR → CI: fmt-check · validate · tflint · security scan
                          · terraform plan (posted to the PR)
              → review code AND plan → merge to main
              → CD: terraform apply of the reviewed plan (prod behind an approval gate)
```

Rule	Why
Nobody applies from a laptop to shared environments	Consistent credentials, logs, reviewed plans
Plan on every PR, apply only after merge	Review sees the exact impact
Apply the **saved plan** (`-out`) or re-plan and compare	No surprises between review and apply
Separate roles for plan (read-only) and apply (write)	Least privilege
Approval gates for production	Human check on destructive changes
Scheduled drift detection (`plan -detailed-exitcode` : exit 2 = changes)	Catch manual changes

GitHub Actions example (OIDC, no stored AWS keys):

```yaml
name: terraform
on:
  pull_request:
    paths: ["envs/dev/**", "modules/**"]
  push:
    branches: [main]
    paths: ["envs/dev/**", "modules/**"]

permissions:
  id-token: write
  contents: read
  pull-requests: write

jobs:
  terraform:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: envs/dev
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.9.8
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-terraform
          aws-region: ap-south-1
      - run: terraform fmt -check -recursive
      - run: terraform init -input=false
      - run: terraform validate
      - run: terraform plan -input=false -out=tfplan
      - name: Apply (main branch only)
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: terraform apply -input=false tfplan
```

Dedicated tools: **Atlantis** (plan/apply via PR comments), **HCP Terraform/Terraform Enterprise**, Spacelift, env0, Scalr (remote runs, policy, cost estimation, drift detection).

9.13 Terraform security

Area	Practice
State	Remote, encrypted, versioned, access-restricted; never in Git; separate state per environment
Secrets	Don't write secrets in `.tf` / `.tfvars`; read them with data sources (Secrets Manager/SSM), generate with `random_password` and store in Secrets Manager, or use managed secrets (`manage_master_user_password = true` on RDS); mark outputs/variables `sensitive`; newer versions support **ephemeral** values that never reach state
Credentials	OIDC/assumed roles, no long-lived keys; separate plan and apply roles; least privilege
Supply chain	Pin provider and module versions; commit the lock file; use trusted module sources
Static analysis	`tflint` (provider-aware lint), **checkov** / **trivy config** / tfsec (misconfigurations: public buckets, open SGs, unencrypted disks)
Policy as code	**OPA/Conftest** on the JSON plan (`terraform show -json tfplan`), HCP Sentinel, AWS Config rules after deployment
Reviews	Every change via PR with plan output; CODEOWNERS for sensitive modules (IAM, networking)

Area	Practice
Protection	`prevent_destroy` on critical resources; deletion protection flags (RDS, ALB)

```
tflint --init && tflint --recursive
checkov -d . --quiet
trivy config .
terraform show -json tfplan > plan.json && conftest test plan.json -p policy/
```

9.14 Testing Terraform

Level	Tool	What it checks
Format/syntax	`terraform fmt -check`, `terraform validate`	Style and internal consistency
Lint	tflint	Invalid instance types, deprecated syntax, naming rules
Security/policy	checkov, trivy, Conftest	Misconfigurations, company rules
Unit/integration	`terraform test` (`.tftest.hcl`)	Assertions on plans or real short-lived deployments
End-to-end	Terratest (Go)	Deploy, call endpoints, destroy

```
# tests/network.tftest.hcl
mock_provider "aws" {} # fake AWS provider: runs offline, no credentials or cost (Terraform 1.7+)

variables {
  name       = "test"
  cidr_block = "10.99.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
}

run "plan_creates_two_public_subnets" {
  command = plan

  assert {
    condition     = length(aws_subnet.public) == 2
    error_message = "Expected one public subnet per AZ."
  }
}
```

`terraform test` (inside the module folder) runs each `run` block. With `mock_provider` it runs offline in seconds (ideal for CI); without it, `command = apply` creates **real** resources and destroys them afterwards.

9.15 Importing existing infrastructure

Many companies start with resources created by hand. Bring them under Terraform without recreating them:

```
import {
  to = aws_s3_bucket.legacy
  id = "acme-legacy-bucket"
}
```

```
terraform plan -generate-config-out=generated.tf   # Terraform writes matching resource code
# review/clean generated.tf, move it into your files
terraform plan                                      # should show "1 to import, 0 to change"
terraform apply
```

After import, `plan` must show **no changes** before you start modifying. Import IDs differ per resource type (bucket name, instance ID, ARN): check each resource's documentation ("Import" section).

9.16 Complete project: modular VPC + load-balanced web tier

Structure:

```
project/
├── modules/network/   main.tf  variables.tf  outputs.tf  versions.tf  tests/
└── envs/dev/          main.tf  (providers, module calls, web tier)
```

modules/network

```
# modules/network/variables.tf
variable "name" {
  type        = string
  description = "Name prefix for all resources"
}

variable "cidr_block" {
  type        = string
  description = "VPC CIDR, e.g. 10.10.0.0/16"
  validation {
    condition     = can(cidrhost(var.cidr_block, 0))
    error_message = "cidr_block must be a valid CIDR."
  }
}

variable "azs" {
  type        = list(string)
  description = "Availability zones, one public and one private subnet each"
}
```

```
# modules/network/main.tf
resource "aws_vpc" "this" {
  cidr_block           = var.cidr_block
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags                 = { Name = "${var.name}-vpc" }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
  tags   = { Name = "${var.name}-igw" }
}

resource "aws_subnet" "public" {
  for_each                = { for i, az in var.azs : az => i }
  vpc_id                  = aws_vpc.this.id
  availability_zone       = each.key
  cidr_block              = cidrsubnet(var.cidr_block, 8, each.value + 1) # .1.0/24, .2.0/24
  map_public_ip_on_launch = true
  tags                    = { Name = "${var.name}-public-${each.key}", Tier = "public" }
}

resource "aws_subnet" "private" {
  for_each          = { for i, az in var.azs : az => i }
  vpc_id            = aws_vpc.this.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(var.cidr_block, 8, each.value + 11) # .11.0/24, .12.0/24
  tags              = { Name = "${var.name}-private-${each.key}", Tier = "private" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
  tags = { Name = "${var.name}-public-rt" }
}

resource "aws_route_table_association" "public" {
  for_each       = aws_subnet.public
  subnet_id      = each.value.id
  route_table_id = aws_route_table.public.id
}
```

```
# modules/network/outputs.tf
output "vpc_id" {
  value = aws_vpc.this.id
}

output "public_subnet_ids" {
  value = [for s in aws_subnet.public : s.id]
}

output "private_subnet_ids" {
  value = [for s in aws_subnet.private : s.id]
}
```

```
# modules/network/versions.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0" # modules use minimum versions; the root module pins exactly
    }
  }
}
```

envs/dev

```
# envs/dev/main.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.70"
    }
  }
  # backend "s3" { ... }  # remote state in real use (9.6)
}

provider "aws" {
  region = "ap-south-1"
  default_tags {
    tags = { Project = "shop", Env = "dev", ManagedBy = "terraform" }
  }
}

locals {
  name = "shop-dev"
}

module "network" {
  source     = "../../modules/network"
  name       = local.name
  cidr_block = "10.10.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
}

data "aws_ssm_parameter" "al2023" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_security_group" "alb" {
  name   = "${local.name}-alb"
  vpc_id = module.network.vpc_id
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "web" {
  name   = "${local.name}-web"
  vpc_id = module.network.vpc_id
  ingress {
    description     = "HTTP only from the ALB"
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
  egress {
    from_port   = 0
    to_port     = 0
```

```
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_lb" "web" {
  name               = "${local.name}-alb"
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb.id]
  subnets            = module.network.public_subnet_ids
}

resource "aws_lb_target_group" "web" {
  name     = "${local.name}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = module.network.vpc_id
  health_check {
    path                = "/"
    healthy_threshold   = 2
    unhealthy_threshold = 2
    interval            = 15
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.web.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.web.arn
  }
}

resource "aws_launch_template" "web" {
  name_prefix            = "${local.name}-web-"
  image_id               = data.aws_ssm_parameter.al2023.value
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data = base64encode(<<-EOT
    #!/bin/bash
    dnf install -y nginx
    echo "<h1>$(hostname)</h1>" > /usr/share/nginx/html/index.html
    systemctl enable --now nginx
  EOT
  )
  metadata_options {
    http_tokens = "required" # IMDSv2
  }
}

resource "aws_autoscaling_group" "web" {
  name             = "${local.name}-web"
  min_size         = 2
  max_size         = 4
  desired_capacity = 2
  # labs: public subnets (no NAT cost); production: private subnets + NAT/endpoints
  vpc_zone_identifier       = module.network.public_subnet_ids
  target_group_arns         = [aws_lb_target_group.web.arn]
  health_check_type         = "ELB"
  health_check_grace_period = 120

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
```

```
  lifecycle {
    ignore_changes = [desired_capacity]
  }
}

output "url" {
  value = "http://${aws_lb.web.dns_name}"
}
```

⚠️ **Cost:** the ALB and two instances are billed hourly. Run `terraform apply`, open the URL (refresh to see both hostnames), then `terraform destroy` the same day. **Production upgrades:** private subnets + one NAT Gateway per AZ (or VPC endpoints), HTTPS listener with ACM, remote state, RDS module, scaling policies, WAF.

9.17 Troubleshooting

Error / symptom	Cause	Fix
`Error acquiring the state lock`	Another run in progress, or a crashed run left the lock	Wait; verify nobody is running; `terraform force-unlock <ID>`
`No valid credential sources found` / `ExpiredToken`	No/expired AWS credentials	`aws sts get-caller-identity`, SSO login, correct profile/role
`AccessDenied` / `UnauthorizedOperation`	Terraform role lacks permissions	Read the action in the error; add least-privilege permission
`resource already exists`	Created outside Terraform or in another state	`import` it, or rename
Plan wants to **replace** something unexpectedly	Changed an attribute that "forces replacement" (e.g. subnet AZ, DB engine)	Read the plan reason; use `moved`, `lifecycle`, or plan a migration
Perpetual diff (changes every plan)	Attribute normalized by the API, or modified by another system	Match the API format; `ignore_changes` for externally managed attributes
`Inconsistent dependency lock file`	Providers changed without updating the lock	`terraform init -upgrade` and commit the lock file
`Provider produced inconsistent result`	Provider bug or eventual consistency	Retry; upgrade provider; check open issues
`Cycle` error	Resources reference each other circularly (e.g. two SGs referencing each other inline)	Use separate `aws_vpc_security_group_ingress_rule` resources
`Invalid count argument` / `for_each` unknown	Count/for_each depends on values known only after apply	Use static keys; split into two applies; derive from inputs
Timeout creating a resource	Slow service (RDS, NAT), health checks failing	Increase `timeouts`; check the resource's events in AWS
Destroy fails (dependency violation)	Resources created outside Terraform still attached (ENIs, objects in a bucket)	Remove them manually or `force_destroy` (careful)

Debug: `TF_LOG=DEBUG terraform plan 2> debug.log`, `terraform console`, `terraform state show`, and CloudTrail for the failing API call.

9.18 Interview questions and answers

#	Question	Answer
1	What is IaC and why use it?	Infrastructure defined as versioned code: repeatable, reviewable, auditable, fast, less drift
2	Declarative vs imperative?	Describe the end state vs write the steps
3	What is a provider?	Plugin translating Terraform resources into a platform's API calls
4	Resource vs data source?	Managed object Terraform creates/updates/deletes vs read-only lookup
5	What is Terraform state and why is it needed?	Mapping between code and real resources (IDs, attributes, dependencies) used to compute changes
6	Why use remote state with locking?	Team sharing, single source of truth, prevents concurrent applies, versioned backups
7	Why is state sensitive?	Contains attribute values including secrets in plain text
8	`terraform plan` vs `apply`?	Preview vs execute changes
9	What does `init` do?	Downloads providers/modules, configures backend, creates the lock file
10	`count` vs `for_each`?	Index-based copies vs keyed copies; for_each is stable when items are added/removed
11	What are modules?	Reusable packages of resources with inputs/outputs; pin versions
12	How do you manage multiple environments?	Separate directories/state per environment with shared modules (or workspaces for simple cases)
13	How do you handle secrets?	Never in code; secrets manager data sources or managed secrets; sensitive/ephemeral values; protect state
14	What is drift and how do you detect it?	Real infra differs from code; `plan` / `-refresh-only` / scheduled drift checks
15	How do you import existing resources?	`import` blocks (+ `-generate-config-out`) or `terraform import`, then plan must show no changes
16	How do you rename a resource without recreating it?	`moved` block (or `terraform state mv`)
17	`prevent_destroy`, `create_before_destroy`, `ignore_changes`?	Block deletion / replace with no downtime / ignore attributes changed outside Terraform
18	What is the lock file?	`.terraform.lock.hcl`: exact provider versions and checksums; commit it
19	How do you pass values between stacks?	Outputs + `terraform_remote_state`, data sources by tags, or SSM parameters
20	How do you run Terraform in CI/CD safely?	fmt/validate/lint/scan + plan on PR, apply after merge with approval, OIDC roles, saved plans

#	Question	Answer
21	What is `depends_on` for?	Explicit ordering when there is no reference between resources
22	What happens if two people apply at once?	With locking, the second waits/fails; without it, state can be corrupted
23	State lock is stuck. What do you do?	Confirm no running apply, then `force-unlock` with the lock ID
24	Terraform vs Ansible?	Terraform provisions infrastructure declaratively with state; Ansible configures systems procedurally/agentlessly
25	Terraform vs CloudFormation?	Multi-cloud with self-managed state vs AWS-only with AWS-managed state
26	What is `terraform test`?	Built-in testing framework with `.tftest.hcl` run blocks and assertions
27	How do you enforce company rules?	Policy as code (OPA/Conftest, Sentinel), checkov/trivy in CI, code owners
28	What causes "forces replacement"?	Changing immutable attributes; Terraform must destroy and recreate
29	How do you structure a large Terraform codebase?	Versioned modules + small per-environment, per-layer stacks with separate states and pipelines
30	What is OpenTofu?	Open-source, community-governed fork of Terraform with a compatible CLI and language

9.19 Self-check: I can…

- [ ] Explain IaC benefits, declarative vs imperative, idempotency, drift and immutable infrastructure.
- [ ] Explain how Terraform core, providers, the graph and state work together.
- [ ] Use every common CLI command and flag; know which files to commit.
- [ ] Write HCL fluently: blocks, references, types, variables with validation, expressions, functions.
- [ ] Configure providers with version constraints, aliases, assume-role and default tags.
- [ ] Set up an S3 remote backend with locking and explain why state is sensitive.
- [ ] Use state commands, refresh-only, and split state by environment/layer.
- [ ] Use data sources, outputs, locals and `terraform_remote_state`.
- [ ] Write and call versioned modules from local, registry and Git sources.
- [ ] Use `count` / `for_each`, `lifecycle`, `moved`, `import`, `removed` and `check` blocks.
- [ ] Structure multi-environment projects and run Terraform safely in CI/CD with OIDC.
- [ ] Secure Terraform: secrets, least privilege, scanning, policy as code.
- [ ] Test with validate, tflint, checkov and `terraform test`.
- [ ] Import existing infrastructure and generate configuration.
- [ ] Build, verify and destroy the VPC + ALB + ASG project.
- [ ] Troubleshoot locks, credentials, replacements, cycles and perpetual diffs.

REMEMBER

- The loop: **write → init → plan → apply**.
- Plan symbols: **+** create · **~** update · **-** destroy · **-/+** replace (check why!).